DIJAGRAM KLASA – SOLID

Grupa 6 – TIM2

SRP (Single Responsibility Principle)

Ovaj princip je ispunjen jer svaka klasa zna samo jedan pojam. Svaka od klasa okvirno izvršava samo zadatke koji su isključivo vezani za tog korisnika ili tu aktivnost.

Primjer tipa takvih klasa bi bile klase: *Klijent, IzvrsilacUsluge(Firma, Majstor), Administrator.* Navedene klase su usko vezane za aktere i direktno oslikavaju mogućnosti koje sami akteri imaju unutar sistema.

Drugi tip klasa su klase: *PonudaZaIzvrsavanjeUsluga, Oglas(OglasZaUslugu, OglasZaRadnoMjesto), PromotivniOglasMajstora, PortfolioSlika, VerifikacijaFirme.* Navedene klase su direktna reprezentacija specifične aktivnosti unutar sistema.

OCP (Open Closed Principle)

Da bi se ovaj princip zadovoljio, korištene su relacije agregacija i kompozicija između nekoliko tipova klasa, kako bi se jasno utvrdili odnosi između cjeline i njenih dijelova, te jasno definisali budući načini nadogradnje postojeće strukture.

Na primjer, uvođenje novog tipa oglasa u sistem ne zahtijeva izmjene klasa: *OglasZaRadnoMjesto* i *OglasZaUslugu,* nego je dovoljno je naslijediti apstraktnu klasu *Oglas.*

LSP (Liskov Substitution Principle)

U dizajnu sistema prikazanom na dijagramu klasa, LSP je zadovoljen kroz korištenje nekoliko apstraktnih klasa, kao što su *ApplicationUser, Oglas* i *IzvrsilacUsluge. ApplicationUser* predstavlja baznu klasu za sve tipve aktera u sistemu, pri čemu sve izvedene klase mogu imati svoje funkcionalnosti i sve se mogu koristiti umjesto osnovnog tipa, tj. bazne klase. Slično se može reći i za preostale dvije pomenute klase.

Dodatno korištenje enumeracija za specifične vrijednosti atributa omogućava standardizovano ponašanje, te se sprječava unos nelegalnih vrijednosti. Osim toga metode odgovaraju ulogama korisnika (klasa), npr. samo administrator može odobriti verifikaciju firme.

ISP (Interface Segregation Principle)

Trenutna struktura projekta ne sadrži interfejs, međutim predviđeno je da se u toku daljnje nadogradnje i implementacije projekta uvede njihovo postojanje u strukturu.

DIP (Dependency Inversion Principle)

Određene klase koje pripadaju modulima visokog nivoa, ne zavise od detalja implementacije, nego zavise od apstrakcije, čime se omogućava fleksibilnost sistema.

Dobar primjer jeste klasa *Recenzija* koja zavisi od apstraktne klase *IzvrsilacUsluge* umjesto od njenih izvedenih tipova tako da se na ovaj način omogućava fleksibilnost da za neke potencijalne naknadne modifikacije izvedenih klasa, ta promijena neće direktno uticati na vezu.